

Embedding Items at Scale: Comparing GNN-Based and ID-Based Item Embeddings in the Yandex Ecosystem

Sergei Makeev
neuralstrg@gmail.com
Yandex
Moscow, Russia

Vladimir Baikarov
nonameuntitled159@gmail.com
Yandex
Moscow, Russia

Artem Matveev
matfu21@ya.ru
Yandex
Moscow, Russia

Kirill Khrylchenko
elighetol@gmail.com
Yandex
Moscow, Russia

Abstract

Transformer-based sequential recommendation models, which process sequences of user-item interactions, rely heavily on the item embedding strategy. Existing approaches either use pretrained item embeddings or learn them end-to-end with the transformer. To the best of our knowledge, no prior work has compared these options from both cost and quality perspectives in a large-scale industrial setting. This paper is a case study that compares pretrained industrial graph neural network item embeddings with end-to-end trainable item embeddings across two mature production recommendation systems at Yandex: Yandex Market and Yandex Music. We additionally evaluate both approaches on a low-resource dataset sampled from Yandex Lavka production logs, for which both the data and code are publicly available for demonstration purposes. Our results show that a separate pretraining stage helps when training data is limited, but provides no worthwhile benefit for large-scale models trained on extensive datasets.

CCS Concepts

• Information systems → Personalization; Learning to rank; Recommender systems.

Keywords

Deep Learning, Personalization, Sequential Recommendation, Graph Representation Learning

1 Introduction

Sequential recommendation aims to predict the next item a user is likely to interact with based on their past interactions. Transformer-based models have been widely adopted for this task in large-scale industrial systems such as Pinterest [6, 14, 19, 20], ByteDance [4, 24], Taobao [5], Kuaishou [13], Zalando [3], Yandex [11], and others. These models take as input a chronologically ordered sequence of user-item interactions and use it to produce personalized recommendations.

One way to encode items in these sequences is to map their IDs to the embeddings learned jointly with the transformer without a separate embedding training stage. However, modern recommender systems operate on large and highly dynamic item catalogs, making it impractical to assign a unique embedding to every item ID. Instead, the hashing trick [18] maps each item ID to an index in a

fixed-size embedding table using a hash function. To reduce collisions, the multihash technique [16] applies multiple hash functions to the same item ID, retrieves several table entries, and combines them by summation [1, 16] or concatenation [7]. Coleman et al. [7] report deploying such multihash embeddings in various web-scale search, ads, and recommendation models at Google.

Another way to encode items is to use pretrained item embeddings. User interactions can be represented as a heterogeneous graph where nodes are users and items, and edges denote relationships such as clicks, purchases, and likes. Graph representation learning aims to learn high-quality node embeddings that can later be used as item embeddings in the transformer model. Pancha et al. [14], Xia et al. [19, 20] report deploying such multi-stage systems at Pinterest. Following Hamilton et al. [10], graph representation learning can be viewed as an encoder-decoder problem, where the encoder maps nodes to embeddings and the decoder reconstructs the graph structure. Graph neural networks (GNNs) can be divided into two groups:

- (1) **Transductive GNNs** directly train node embeddings and require all nodes to be present during training. As new nodes appear, such models must be retrained. Albeit simple, TwHIN is an efficient model of this type, proposed by El-Kishky et al. [8] and deployed at X (formerly Twitter). The encoder maps the set of nodes $\mathcal{V}$ and relations $\mathcal{R}$ to trained embeddings: $\text{ENC}(\cdot) = \theta$, $\theta \in \mathbb{R}^d$, where d is the embedding dimension. Thus, the trained parameters can be viewed as an embedding matrix $\Theta \in \mathbb{R}^{(|\mathcal{V}|+|\mathcal{R}|) \times d}$. The decoder reconstructs the probability logits of an edge $e = (s, r, t)$, formed by two nodes $s, t \in \mathcal{V}$ and a relation $r \in \mathcal{R}$, being present in the graph: $\text{DEC}(e) = \text{DEC}(\text{ENC}(s), \text{ENC}(r), \text{ENC}(t)) = (\theta_s + \theta_r)^T \theta_t$. The model is trained to maximize the log-likelihood of predicting a binary "real" or "fake" label for observed edges in a heterogeneous graph $\mathcal{G}$ and negatively sampled edges $\mathcal{N}(\cdot)$:

$$\arg \max_{\Theta} \sum_{e \in \mathcal{G}} \left[\log \sigma(\text{DEC}(e)) + \sum_{e' \in \mathcal{N}(e)} \log \sigma(-\text{DEC}(e')) \right],$$

where $\mathcal{N}(s, r, t) = \{(s, r, t') : t' \in \mathcal{V}\} \cup \{(s', r, t) : s' \in \mathcal{V}\}$ is the set of negative edges obtained by replacing either the source or the target node of the positive edge.

- (2) **Inductive GNNs** learn a generalizable, parameterized function rather than embeddings for specific nodes. This function aggregates information from a node local neighbors based on their features. As a result, the full graph is not required during training, and previously unseen nodes can be handled naturally. MultiBiSage [9] extends PinSage [23], which was used for item embeddings in Pinterest production models [14, 19, 20], and adapts it to heterogeneous graphs. A heterogeneous graph $\mathcal{G}$ can be decomposed into multiple bipartite graphs $\mathcal{G}_r$, each containing a single relation type $r \in \mathcal{R}$. In MultiBiSage, the encoder processes visual and textual features of a node $s \in \mathcal{V}$ and its neighbors in each bipartite graph $\mathcal{G}_r$ using a transformer to obtain intermediate representations. These representations are then aggregated across all bipartite graphs by another transformer to form a final embedding e_s^θ , where θ denotes encoder parameters. The decoder learns the probability distribution $p(t | s)$, $t, s \in \mathcal{V}$, of node t being "similar" to s over the item catalog using a sampled softmax:

$$\arg \max_{\theta} \sum_{(s,t)} \log \left(\frac{\exp\{\langle e_s^\theta, e_t^\theta \rangle - \log Q(t)\}}{\sum_{t' \in \mathcal{N}} \exp\{\langle e_s^\theta, e_{t'}^\theta \rangle - \log Q(t')\}} \right),$$

where $\mathcal{N}$ is a set of in-batch and uniformly sampled negatives t' , known as Mixed Negative Sampling [21], and $\log Q(t)$ is the logQ correction term [22].

This paper presents a case study comparing pretrained TwHIN and MultiBiSage item embeddings (referred to as **GNN embeddings**) with multihash item ID embeddings trained end-to-end with the transformer from scratch (referred to as **ID embeddings**) across two mature recommendation systems at Yandex, as well as on a public dataset, to extend the conclusions to low-resource scenarios. We analyze both cost and quality, and assess whether the additional pretraining stage is worthwhile.

2 Method

Sequential recommendation aims to recommend the most relevant item to a user based on their interaction history $x^u = (i_1^u, i_2^u, \dots, i_{l_u}^u)$, where l_u is the number of interactions for user u .

In our experiments, we focus on ranking models. Following the production ranking setup proposed by Khrylchenko and Fritzler [11], we split training of the transformer ranker into pretraining and fine-tuning stages. In both stages, the user is represented by their interaction history x^u .

Model architecture. We use a production two-tower architecture, where the user tower is a transformer over the interaction sequence x^u , and the item tower is a residual network.

The item tower combines the item title with the item embedding (GNN or ID embedding). An embedding bag layer maps the BPE-tokenized title into token embeddings and sums them. The BPE vocabulary contains $O(10^5)$ tokens. The item embedding is added to the title representation. A linear layer then projects the resulting embedding into a higher-dimensional space, followed by three residual blocks. Each block consists of a linear layer, ReLU activation, dropout, and Layer Normalization. A final linear layer

maps the embedding back to the original dimension, and the output is L_2 -normalized.

The user tower encodes the user interaction history. First, the item tower is applied to each item in the interaction sequence. Learnable action embeddings (the sets of possible actions are platform-dependent) and positional embeddings are added element-wise, and a CLS token is concatenated to the sequence. A bidirectional transformer encoder is then applied, and the L_2 -normalized output corresponding to the CLS token is used as the user representation.

A dot product between user and item embeddings, equivalent to cosine similarity, is used as the relevance score.

Pretraining is formulated as a next-item prediction task. For each k , given a subsequence $(i_1^u, i_2^u, \dots, i_{k-1}^u)$, the model predicts i_k^u . A sampled softmax loss with in-batch negatives is used to learn a probability distribution over the item catalog:

$$\mathcal{L}_{\text{pretrain}}(u, i_k^u, \mathcal{N}) = -\log \frac{\exp\{r_{ui_k^u}/\tau\}}{\exp\{r_{ui_k^u}/\tau\} + \sum_{n \in \mathcal{N}} \exp\{r_{un}/\tau\}},$$

where r_{ui} is the relevance score between user u and item i , $\mathcal{N}$ is a set of in-batch negatives, and τ is a temperature parameter.

Fine-tuning trains the model to rank candidate items. For each user request, the recommender system returns a set of s^u items $\mathcal{I}^u = \{i_1^u, \dots, i_{s^u}^u\}$ shown to user u . For each of these items, user actions $\mathcal{A}^u = \{a_1^u, \dots, a_{s^u}^u\}$ are stored and used to calculate both pointwise and pairwise ranking losses.

Pointwise ranking is formulated as a click prediction task:

$$\mathcal{L}_{pt}(u, i) = -t_{ui} \log f_{ui}^{pt} - (1 - t_{ui}) \log(1 - f_{ui}^{pt}),$$

where $f_{ui}^{pt} = \sigma(\alpha \cdot r_{ui} + \beta)$, α, β are trained parameters, r_{ui} is the relevance score, and $t_{ui} = \mathbb{1}\{a_i^u = \text{"click"}\}$.

For pairwise ranking, we construct all pairs (i_p^u, i_n^u) between items i_p^u we treat as positive interactions and items i_n^u with which the user did not interact. Following Bai et al. [2], the pairwise loss is:

$$\mathcal{L}_{pr}(u, i_p^u, i_n^u) = -\log \frac{f_{up}^{pr}}{f_{up}^{pr} + f_{un}^{pr}},$$

where $f_{ui}^{pr} = \sigma(\delta \cdot r_{ui} + \gamma)$, and δ, γ are trained parameters.

The fine-tuning loss is a weighted sum of pointwise and pairwise losses:

$$\mathcal{L}_{\text{finetune}}(u, \mathcal{I}^u, \mathcal{A}^u) = \sum_{i \in \mathcal{I}^u} \mathcal{L}_{pt}(u, i) + \frac{1}{10} \sum_{(i_p^u, i_n^u)} \mathcal{L}_{pr}(u, i_p^u, i_n^u).$$

To compare GNN and ID embeddings, we plug each embedding type into the item tower and train the transformer model. The fine-tuned model outputs ranking scores, which are then used, together with other statistical features, as input to a CatBoost [15] production ranker. To evaluate the contribution of the transformer model, we train the CatBoost ranker twice: once with these ranking scores and once without them, and report the relative difference in metrics. In all experiments, we use a timestamp-based strategy to split the training and test data.

To assess statistical significance, we split the CatBoost dataset into 32 equal folds. For each fold, we train both the baseline CatBoost and the CatBoost with the additional feature on that fold and compute metrics on the remaining data. We compare the results

using the Wilcoxon signed-rank test and report differences that are significant at $p < 0.01$.

3 Experiments

We compare pretrained GNN item embeddings, trained before the transformer ranker, with embeddings trained end-to-end from scratch (ID embeddings) on three Yandex platforms: Yandex Market (e-commerce), Yandex Music (music streaming), and Yandex Lavka (grocery e-commerce). For Yandex Market and Yandex Music, we experiment with production models described in [11, 12] and large-scale in-house datasets to draw conclusions for large-scale models. For Yandex Lavka, we train a smaller model on a low-resource dataset; both the data and code are available on our GitHub (see subsection 3.3). When comparing GNN and ID embeddings, we obtain them for the same set of items.

3.1 Yandex Market

We formulate the following research questions:

- **RQ1:** Do pretrained GNN embeddings lead to better ranking quality than ID embeddings in a large-scale domain?
- **RQ2:** Does combining both embedding approaches provide worthwhile improvements?

Data. We collect user logs over a one-year period from Yandex Market, which serves millions of users. The data contains $O(10^7)$ items, and user actions include clicks, cart additions, likes, and purchases. Test data is collected over two days following the training period.

We use TwHIN and MultiBiSage to train GNN item embeddings. Embeddings are computed for the 17% most popular items, which cover most user interactions; all other items share a single embedding.

TwHIN. We construct a heterogeneous user-item graph with edges representing clicks, orders, likes, and cart additions.

MultiBiSage. Here, we construct two bipartite graphs: an item-cart graph (connecting items to the carts they were added to) and an item-order graph (connecting items to completed orders). To reduce popularity bias, we prune the graphs by limiting the number of edges per node to at most several thousand. We also exclude cart and order nodes whose normalized entropy exceeds a threshold. Entropy is computed over item categories, and normalization is performed with respect to cart or order size, rather than the number of categories. For each bipartite graph, we use a 4-layer transformer over visual and textual representations. We then aggregate vectors from the two graphs with a 2-layer transformer. Following Gurukar et al. [9], we sample 50 neighbors per node using random walks to define the local neighborhood. We then construct positive item pairs, where the source item is the item viewed by the user and the target item is the item clicked from the similar-items recommendations.

ID embeddings. We use an embedding matrix with $O(10^6)$ entries and 64 dimensions. For the multihash technique, we use 6 lookups. The corresponding embeddings are concatenated and projected to the initial dimension.

The transformer processes sequences of up to 256 events. To address RQ2, we combine TwHIN and ID embeddings in the item tower and study their joint effect. We report offline relative nDCG

Table 1: Relative nDCG difference on large-scale Yandex Market data and production model. Results are statistically significant.

Item embeddings	Discovery	Cart	Retargeting
No (only content info)	+0.506%	+0.103%	+0.565%
TwHIN	+0.790%	+0.151%	+0.943%
MultiBiSage	+0.565%	+0.122%	+0.651%
ID embeddings	+1.238%	+0.215%	+1.486%
TwHIN + ID embeddings	+1.273%	+0.235%	+1.522%

differences for three recommendation surfaces: retargeting, discovery, and recommendations from the cart page. Retargeting is an unconstrained personalization setting, similar to eBay’s Recently Viewed Items module [17], while discovery restricts recommendations to previously unseen items.

The results in Table 1 show that ID embeddings trained end-to-end from scratch outperform pretrained GNN embeddings in this large-scale setting. Combining GNN and ID embeddings yields additional gains, but we do not consider the extra GNN training cost to be worthwhile.

3.1.1 Compute Time and Memory Analysis. All experiments were conducted on 8 NVIDIA A100 GPUs rented via a third-party compute provider. For ID embeddings, training time depends on embedding dimensionality, number of lookups, and embedding table size. It ranges from 40 hours (16 dimensions, 2 lookups) to 70 hours (256 dimensions, 6 lookups), and from 41 hours for $O(10^4)$ embedding table entries to 52 hours for $O(10^6)$ entries in our experimental setup (64 dimensions, 6 lookups). MultiBiSage training takes 52 hours, while TwHIN training takes 5 hours.

During the main model training, we store frozen GNN embeddings on SSD using Lightning Memory-Mapped Database (LMDB), while learnable ID embeddings are kept on GPU. Due to additional RAM-GPU communication overhead, we do not report training time changes from using GNN embeddings.

3.2 Yandex Music

We formulate the following research questions:

- **RQ3:** Do the comparison results between the two embedding approaches generalize to another large-scale domain?
- **RQ4:** Does fine-tuning pretrained GNN item embeddings improve performance?

Data. The dataset contains hundreds of billions of interactions from $O(10^7)$ users and over $O(10^6)$ tracks. User actions include listening time, likes, and skips.

We train embeddings for the same $O(10^5)$ most popular tracks, which account for more than 90% of user interactions. Since textual and visual content is less informative for tracks than in e-commerce, and given the weak MultiBiSage results in the previous experiment, we only train TwHIN item embeddings.

TwHIN. In the graph, nodes are users and tracks, and edges correspond to likes and long plays (at least 90% of the track played).

Table 2: Relative accuracy difference on large-scale Yandex Music data and production model. Results are statistically significant.

Item embeddings	Embeddings are fine-tuned	Pair Accuracy	Weighted Pair Accuracy
TwHIN	-	+0.348%	+0.325%
TwHIN	✓	<u>+0.524%</u>	<u>+0.448%</u>
ID embeddings	✓	+0.699%	+0.603%

ID embeddings. Since we only embed $O(10^5)$ items, we train unique embeddings for all of them without hashing.

We train the production transformer ranker [12] on sequences of up to 2048 interactions. We report the relative differences in pair accuracy and weighted pair accuracy, where the latter reweights actions based on the importance of each action type. To answer RQ4, instead of combining both embedding strategies, we fine-tune pre-trained TwHIN item embeddings end-to-end with the transformer model.

As shown in Table 2, ID embeddings significantly outperform TwHIN embeddings. Interestingly, even after fine-tuning, TwHIN embeddings do not improve the final performance.

3.3 Yandex Lavka

For this experiment, we release data and code on our GitHub¹. We investigate the following research question:

- **RQ5:** Does training GNN item embeddings yield improvements in a limited-resource setting?

Data. The data, sampled over a one-year period, comes from the Yandex Lavka grocery e-commerce service. The dataset contains 15 million user-item interactions from 3315 users and 25833 items. Available actions are views, clicks, cart additions, and purchases.

To keep the published dataset size manageable, we do not include textual or image descriptions of items and restrict our study to TwHIN GNN embeddings. Due to the relatively small number of unique items, we train both *TwHIN* and *ID embeddings* for all items, without popularity-based clipping. We train TwHIN on a user-item graph, where edges correspond to clicks, cart additions, and purchases.

The ranking transformer processes sequences of up to 256 events. We report nDCG@5, nDCG@10, and nDCG@20 per user request. Unlike the previous experiments, transformer scores are used directly to rank items and are not passed as additional CatBoost features.

In contrast to the previous experiments, the results in Table 3 show that models using pretrained TwHIN embeddings consistently outperform those using only ID embeddings. This indicates that when training data is limited, an additional GNN training stage improves model performance.

Table 3: Experiment results on low-resource Yandex Lavka data and small transformer model. Results are averaged over 10 runs.

Item embeddings	Embeddings are fine-tuned	nDCG @5	nDCG @10	nDCG @20
TwHIN	-	<u>0.337</u>	<u>0.409</u>	<u>0.457</u>
TwHIN	✓	0.342	0.415	0.464
ID embeddings	✓	0.333	0.406	0.456

4 Conclusion

We compared two item embedding approaches for ranking: pre-trained graph neural network embeddings and embeddings trained end-to-end with the transformer from scratch. We ran experiments on three Yandex platforms: Yandex Market, Yandex Music, and Yandex Lavka. For Yandex Market and Yandex Music, we used production models and large-scale training datasets. For Yandex Lavka, we used a smaller dataset and a compact model, and released both the dataset and code. Our results show that in low-resource settings, pretraining item embeddings is beneficial, while in large-scale settings pretrained embeddings do not provide benefits that justify their additional cost.

References

- [1] Prabhat Agarwal, Minhazul Islam Sk, Nikil Pancha, Kurchi Subhra Hazra, Jiajing Xu, and Chuck Rosenberg. 2024. OmniSearchSage: Multi-Task Multi-Entity Embeddings for Pinterest Search. In *Companion Proceedings of the ACM on Web Conference 2024*. 121–130.
- [2] Aijun Bai, Rolf Jagerman, Zhen Qin, Le Yan, Pratyush Kar, Bing-Rong Lin, Xuanhui Wang, Michael Bendersky, and Marc Najork. 2023. Regression compatible listwise objectives for calibrated ranking with binary relevance. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4502–4508.
- [3] Marjan Celiklik, Ana Peleteiro Ramallo, and Jacek Wasilewski. 2022. Reusable self-attention recommender systems in fashion industry applications. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 448–451.
- [4] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. Longer: Scaling up long sequence modeling in industrial recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 247–256.
- [5] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st international workshop on deep learning practice for high-dimensional sparse data*. 1–4.
- [6] Xiangyi Chen, Kousik Rajesh, Matthew Lawhon, Zelun Wang, Hanyu Li, Haomiao Li, Saurabh Vishwas Joshi, Pong Eksombatchai, Jaewon Yang, Yi-Ping Hsu, et al. 2025. Pinfm: foundation model for user activity sequences at a billion-scale visual discovery platform. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 381–390.
- [7] Benjamin Coleman, Wang-Cheng Kang, Matthew Fahrbach, Ruoxi Wang, Lichan Hong, Ed Chi, and Derek Cheng. 2023. Unified Embedding: Battle-tested feature representations for web-scale ML systems. *Advances in Neural Information Processing Systems* 36 (2023), 56234–56255.
- [8] Ahmed El-Kishky, Thomas Markovich, Serim Park, Chetan Verma, Baekjin Kim, Ramy Eskander, Yuri Malkov, Frank Portman, Sofia Samaniego, Ying Xiao, et al. 2022. TwHIN: Embedding the twitter heterogeneous information network for personalized recommendation. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 2842–2850.
- [9] Saket Gururkar, Nikil Pancha, Andrew Zhai, Eric Kim, Samson Hu, Srinivasan Parthasarathy, Charles Rosenberg, and Jure Leskovec. 2022. Multibisage: A web-scale recommendation system using multiple bipartite graphs at pinterest. *arXiv preprint arXiv:2205.10666* (2022).
- [10] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation learning on graphs: Methods and applications. *arXiv preprint arXiv:1709.05584* (2017).
- [11] Kirill Khrylchenko and Alexander Fritzler. 2023. Personalized transformer-based ranking for e-commerce at yandex. *arXiv preprint arXiv:2310.03481* (2023).

¹<https://github.com/matfu-pixel/Comparing-GNN-based-and-ID-based-Item-Embeddings-Web-Scale-Perspective>

- [12] Kirill Khrylchenko, Artem Matveev, Sergei Makeev, and Vladimir Baikalov. 2025. Scaling recommender transformers to one billion parameters. *arXiv preprint arXiv:2507.15994* (2025).
- [13] Chi Liu, Jiangxia Cao, Rui Huang, Kai Zheng, Qiang Luo, Kun Gai, and Guorui Zhou. 2024. KuaiFormer: Transformer-Based Retrieval at Kuaishou. *arXiv preprint arXiv:2411.10057* (2024).
- [14] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.
- [15] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: unbiased boosting with categorical features. *Advances in neural information processing systems* 31 (2018).
- [16] Dan Tito Svenstrup, Jonas Hansen, and Ole Winther. 2017. Hash embeddings for efficient word representations. *Advances in neural information processing systems* 30 (2017).
- [17] Tian Wang, Yuri M. Brovman, and Sriganesh Madhvanath. 2021. Personalized embedding-based e-commerce recommendations at ebay. *arXiv preprint arXiv:2102.06156* (2021).
- [18] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. 2009. Feature hashing for large scale multitask learning. In *Proceedings of the 26th annual international conference on machine learning*. 1113–1120.
- [19] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.
- [20] Xue Xia, Saurabh Joshi, Kousik Rajesh, Kangnan Li, Yangyi Lu, Nikil Pancha, Dhruvil Badani, Jiajing Xu, and Pong Eksombatchai. 2025. TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6881–6882.
- [21] Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, Lichan Hong, Yang Li, Simon Xiaoming Wang, Taibai Xu, and Ed H Chi. 2020. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion proceedings of the web conference 2020*. 441–447.
- [22] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. 2019. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM conference on recommender systems*. 269–277.
- [23] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
- [24] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. *arXiv preprint arXiv:2510.26104* (2025).